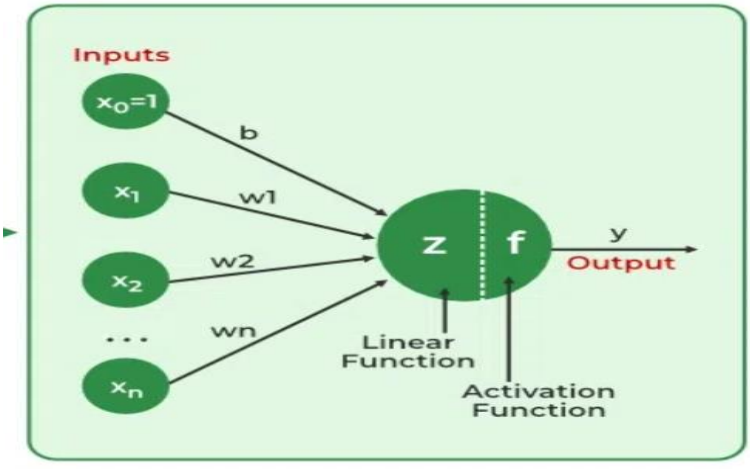
Unit-7: Neural Networks

Neural Networks: Neural Networks are computational models that mimic the complex functions of the human brain. The neural networks consist of interconnected nodes or neurons that process and learn from data, enabling tasks such as pattern recognition and decision making in machine learning.

Neural networks extract identifying features from data, lacking pre-programmed understanding. Network components include neurons, connections, weights, biases, propagation functions, and a learning rule. Neurons receive inputs, governed by thresholds and activation functions. Connections involve weights and biases regulating information transfer. Learning, adjusting weights and biases, occurs in three stages: input computation, output generation, and iterative refinement enhancing the network's proficiency in diverse tasks. These include:

- The neural network is simulated by a new environment.
- Then the free parameters of the neural network are changed as a result of this simulation.
- The neural network then responds in a new way to the environment because of the changes in its free parameters.



Basic Structure of a Neural Network

Neurons: The basic units of a neural network, similar to the brain's neurons, where each neuron receives inputs, processes them, and produces an output.

Layers:

Input Layer: The first layer that receives the initial data.

Hidden Layers: Intermediate layers that process the inputs. There can be multiple hidden layers in a deep neural network.

Output Layer: The final layer that produces the output of the network.

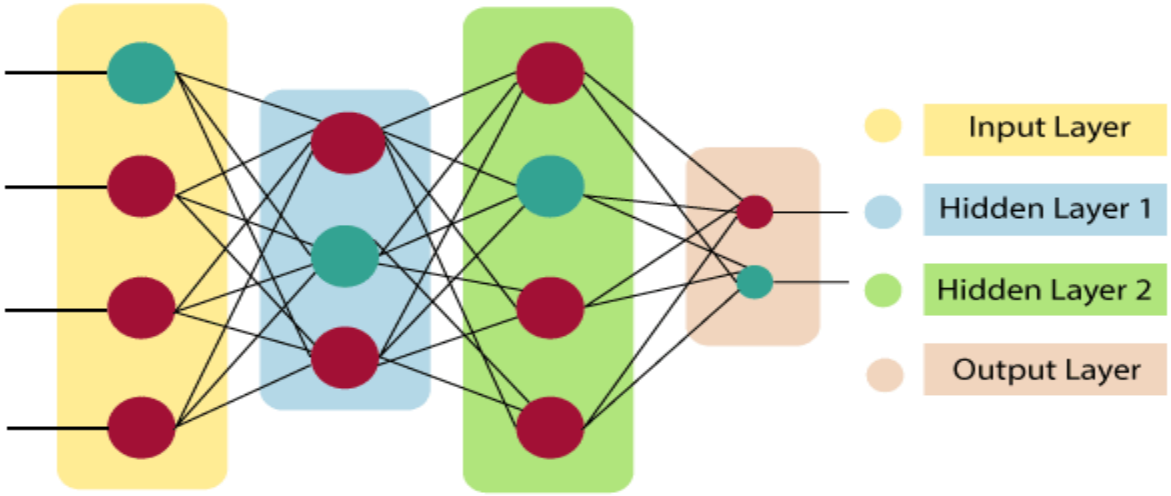


Fig: Structure of Neural Network

How Neurons Work?

Inputs and Weights: Each neuron receives one or more inputs, each of which is multiplied by a weight, a parameter that the network adjusts during training.

Summation: The weighted inputs are summed up.

Activation Function: The sum is passed through an activation function, which introduces non-linearity into the model and helps the network learn complex patterns. Common activation functions include sigmoid, ReLU (Rectified Linear Unit), and tansh(hyperbolic function).

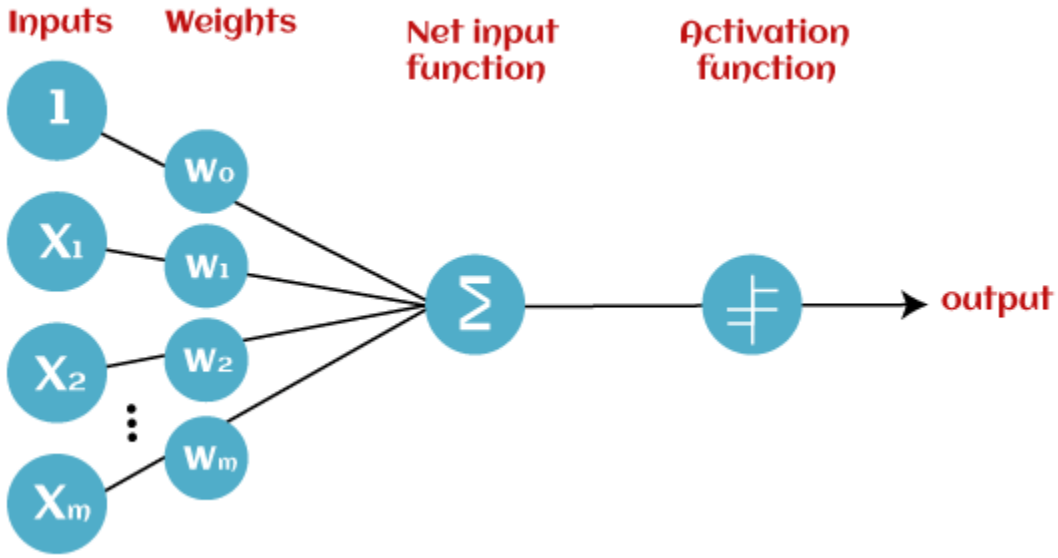
Output: The result after applying the activation function is the neuron's output, which can be passed to the neurons in the next layer.

Perceptron:

Perceptron is one of the first and most straightforward models of artificial neural networks. It was introduced by Frank Rosenblatt in 1957s. It is the simplest type of feedforward neural network, consisting of a single layer of input nodes that are fully connected to a layer of output nodes. It can learn the linearly separable patterns. It uses slightly different types of artificial neurons known as threshold logic units (TLU). It was first introduced by McCulloch and Walter Pitts in the 1940s.

Basic Components of Perceptron

Mr. Frank Rosenblatt invented the perceptron model as a binary classifier which contains three main components. These are as follows:



Input Nodes or Input Layer:

This is the primary component of Perceptron which accepts the initial data into the system for further processing. Each input node contains a real numerical value.

Weight and Bias:

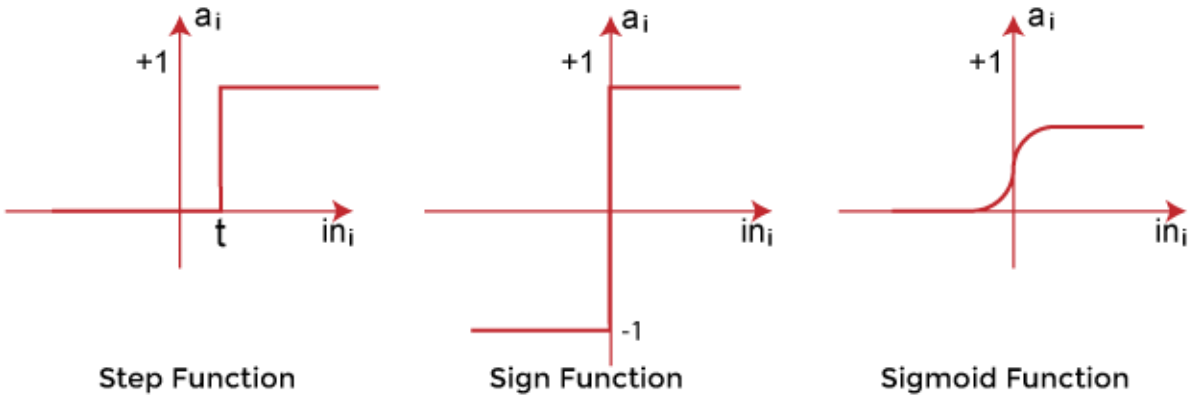
Weight parameter represents the strength of the connection between units. This is another most important parameter of Perceptron components. Weight is directly proportional to the strength of the associated input neuron in deciding the output. Further, Bias can be considered as the line of intercept in a linear equation.

Activation Function:

These are the final and important components that help to determine whether the neurons will fire or not. Activation Function can be considered primarily as a step function.

Types of Activation functions:

- Sign function
- Step function, and
- Sigmoid function



Step Function Sign Function Sigmoid Function

The data scientist uses the activation function to take a subjective decision based on various problem statements and forms the desired outputs. Activation function may differ (e.g., Sign, Step, and Sigmoid) in perceptron models by checking whether the learning process is slow or has vanishing or exploding gradients.

Types of Perceptron Models

Based on the layers, Perceptron models are divided into two types. These are as follows:

- Single-layer Perceptron Model
- Multi-layer Perceptron model

Single Layer Perceptron Model:

This is one of the easiest artificial neural networks (ANN) types. A single-layered perceptron model consists feed-forward network and also includes a threshold transfer function inside the model. The main objective of the single-layer perceptron model is to analyze the linearly separable objects with binary outcomes.

In a single layer perceptron model, its algorithms do not contain recorded data, so it begins with inconstantly allocated input for weight parameters. Further, it sums up all inputs (weight). After adding all inputs, if the total sum of all inputs is more than a pre-determined value, the model gets activated and shows the output value as +1.

If the outcome is same as pre-determined or threshold value, then the performance of this model is stated as satisfied, and weight demand does not change. However, this model consists of a few discrepancies triggered when multiple weight inputs values are fed into the model. Hence, to find desired output and minimize errors, some changes should be necessary for the weights input.

"Single-layer perceptron can learn only linearly separable patterns."

Multi-Layered Perceptron Model:

Like a single-layer perceptron model, a multi-layer perceptron model also has the same model structure but has a greater number of hidden layers.

The multi-layer perceptron model is also known as the Backpropagation algorithm, which executes in two stages as follows:

Forward Stage: Activation functions start from the input layer in the forward stage and terminate on the output layer.

Backward Stage: In the backward stage, weight and bias values are modified as per the model's requirement. In this stage, the error between actual output and demanded originated backward on the output layer and ended on the input layer.

Hence, a multi-layered perceptron model has considered as multiple artificial neural networks having various layers in which activation function does not remain linear, similar to a single layer perceptron model. Instead of linear, activation function can be executed as sigmoid, TanH, ReLU, etc., for deployment.

A multi-layer perceptron model has greater processing power and can process linear and non-linear patterns. Further, it can also implement logic gates such as AND, OR, XOR, NAND, NOT, XNOR, NOR.

Advantages of Multi-Layer Perceptron:

- A multi-layered perceptron model can be used to solve complex non-linear problems.
- It works well with both small and large input data.
- It helps us to obtain quick predictions after the training.
- It helps to obtain the same accuracy ratio with large as well as small data.

Disadvantages of Multi-Layer Perceptron:

- In Multi-layer perceptron, computations are difficult and time-consuming.
- In multi-layer Perceptron, it is difficult to predict how much the dependent variable affects each independent variable.
- The model functioning depends on the quality of the training.

Adaline and Madaline Network

An artificial neural network inspired by the human neural system is a network used to process the data which consist of three types of layer i.e input layer, the hidden layer, and the output layer. The basic neural network contains only two layers which are the input and output layers. The layers are connected with the weighted path which is used to find net input data. In this section, we will discuss two basic types of neural networks Adaline which doesn't have any hidden layer, and Madaline which has one hidden layer.

Adaline (Adaptive Linear Neural):

A network with a single linear unit is called Adaline (Adaptive Linear Neural). A unit with a linear activation function is called a linear unit. In Adaline, there is only one output unit and output values are bipolar (+1,-1). Weights between the input unit and output unit are adjustable. It uses the delta rule

i.e w_i (new)= w_i (old) + $(t-y_{in})x_i$, where w_i , y_{in} and t are the weight, predicted output, and true value respectively.

The learning rule is found to minimize the mean square error between activation and target values. Adaline consists of trainable weights, it compares actual output with calculated output, and based on error training algorithm is applied.

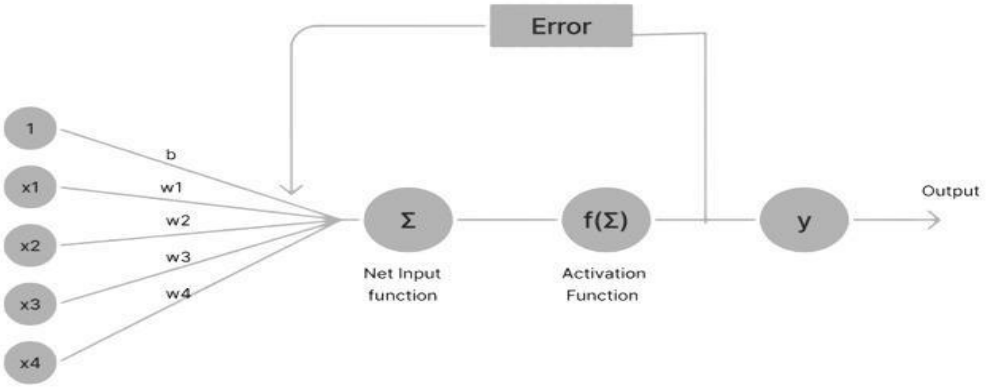
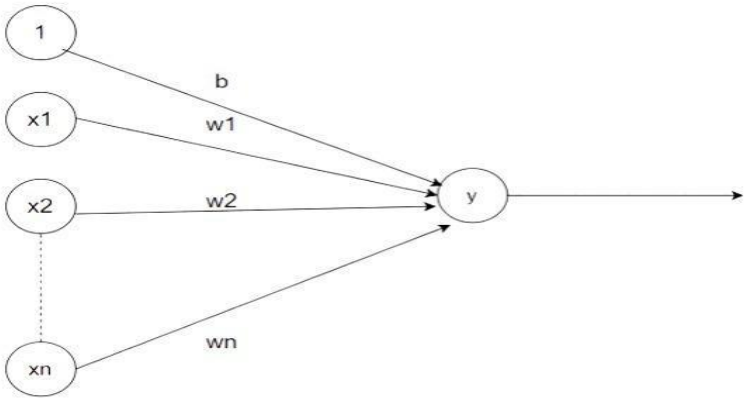


Fig: Workflow Adaline

First, calculate the net input to your Adaline network then apply the activation function to its output then compare it with the original output if both the equal, then give the output else send an error back to the network and update the weight according to the error which is calculated by the delta learning rule. i.e;

which is calculated by the delta learning rule. i.e i.e w_i (new)= w_i (old) + $(t-y_{in})x_i$, where w_i , y_{in} and t are the weight, predicted output, and true value respectively.



In Adaline, all the input neuron is directly connected to the output neuron with the weighted connected path. There is a bias b of activation function 1 is present.

Madaline (Multiple Adaptive Linear Neuron):

The Madaline (supervised Learning) model consists of many Adaline in parallel with a single output unit. The Adaline layer is present between the input layer and the Madaline layer hence Adaline layer is a hidden layer. The weights between the input layer and the hidden layer are adjusted, and the weight between the hidden layer and the output layer is fixed.

It may use the majority vote rule, the output would have an answer either true or false. Adaline and Madaline layer neurons have a bias of '1' connected to them. Use of multiple Adaline helps counter the problem of non-linear separability.

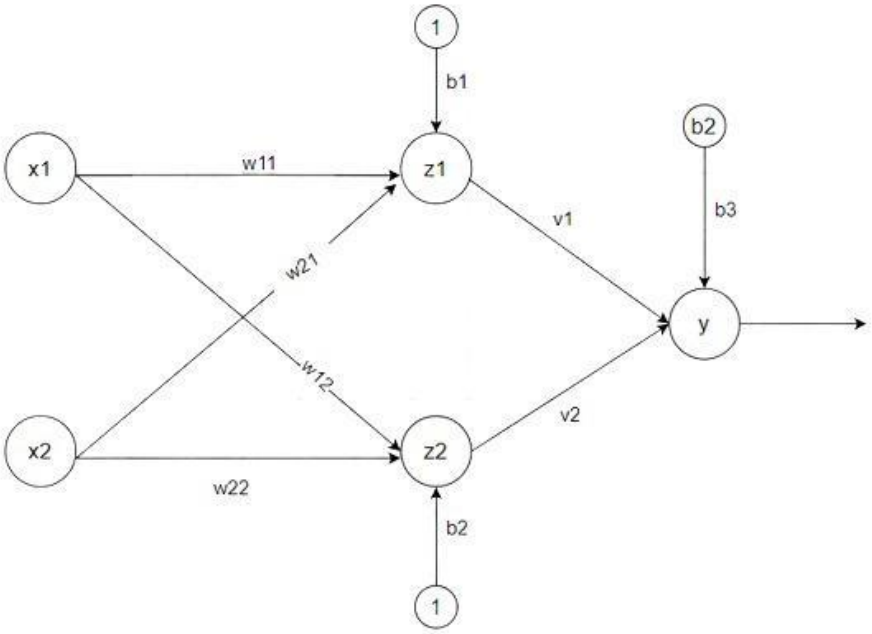


Fig: Madaline Architecture

There are three types of a layer present in Madaline First input layer contains all the input neurons, the Second hidden layer consists of an adaline layer, and weights between the input and hidden layers are adjustable and the third layer is the output layer the weights between hidden and output layer is fixed they are not adjustable.

Back propagation

Backpropagation is an effective algorithm used to train artificial neural networks, especially in feed-forward neural networks. Backpropagation is an iterative algorithm, that helps to minimize the cost function by determining which weights and biases should be adjusted. During every epoch, the model learns by adapting the weights and biases to minimize the loss by moving down toward the gradient of the error. Thus, it involves the two most popular optimization algorithms, such as gradient descent or stochastic gradient descent.

Computing the gradient in the backpropagation algorithm helps to minimize the cost function and it can be implemented by using the mathematical rule called chain rule from calculus to navigate through complex layers of the neural network.

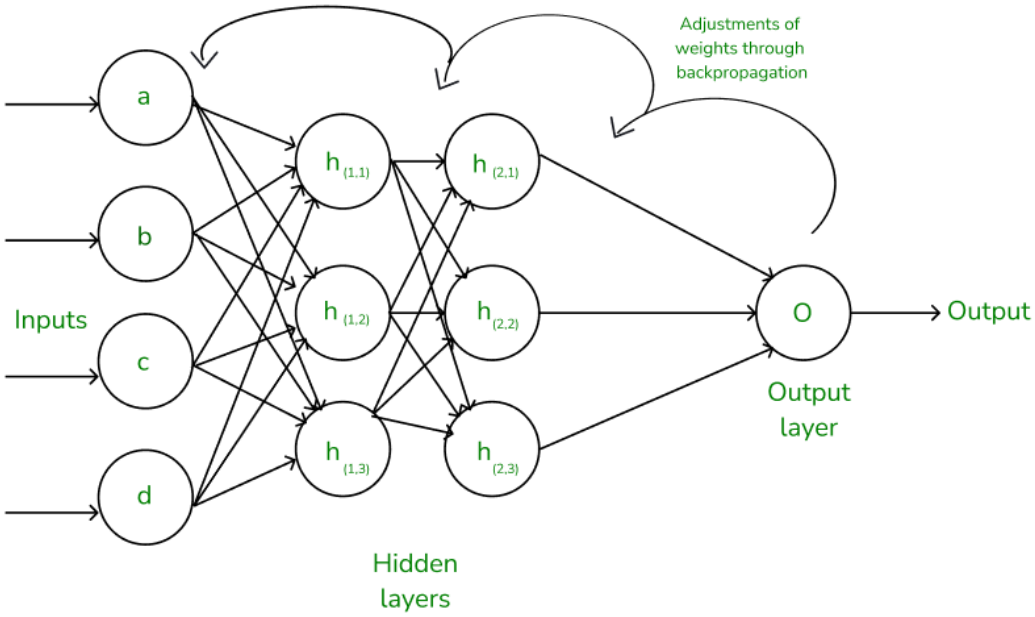


Fig: A simple illustration of how the backpropagation works

Hopfield Neural Network

The Hopfield Neural Networks, invented by Dr John J. Hopfield consists of one layer of 'n' fully connected recurrent neurons. It is generally used in performing auto-association and optimization tasks. It is calculated using a converging interactive process and it generates a different response than our normal neural nets.

Discrete Hopfield Network

It is a fully interconnected neural network where each unit is connected to every other unit. It behaves in a discrete manner, i.e. it gives finite distinct output, generally of two types:

Binary (0/1)

Bipolar (-1/1)

The weights associated with this network are symmetric in nature and have the following properties.

1. $w_{ij} = w_{ji}$
2. $w_{ii} = 0$

Structure & Architecture of Hopfield Network

- Each neuron has an inverting and a non-inverting output.
- Being fully connected, the output of each neuron is an input to all other neurons but not the self.

The below figure shows a sample representation of a Discrete Hopfield Neural Network architecture having the following elements.

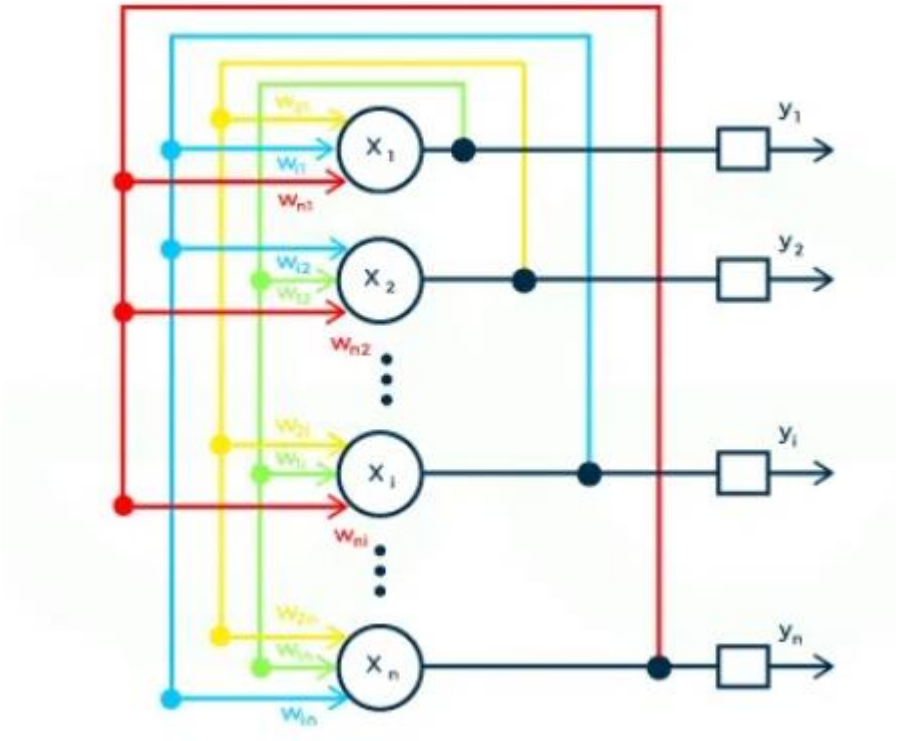


Fig: Discrete Hopfield Network Architecture

[x1 , x2 , ... , xn] -> Input to the n given neurons.

[y1 , y2 , ... , yn] -> Output obtained from the n given neurons

W_{ij} -> weight associated with the connection between the i^{th} and the j^{th} neuron.

Boltzmann Machines:

Boltzmann Machines is an unsupervised DL model in which every node is connected to every other node. That is, unlike the ANNs, CNNs, RNNs and SOMs, the Boltzmann Machines are undirected (or the connections are bidirectional). Boltzmann Machine is not a deterministic DL model but a stochastic or generative DL model. It is rather a representation of a certain system. There are two types of nodes in the Boltzmann Machine-Visible nodes-those nodes which we can and do measure, and the Hidden nodes (those nodes which we cannot or do not measure). Although the node types are different, the Boltzmann machine considers them as the same and everything works as one single system. The training data is fed into the Boltzmann Machine and the weights of the system are adjusted accordingly. Boltzmann machines help us understand abnormalities by learning about the working of the system in normal conditions.

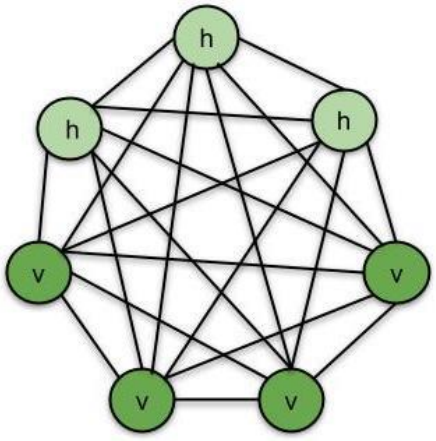


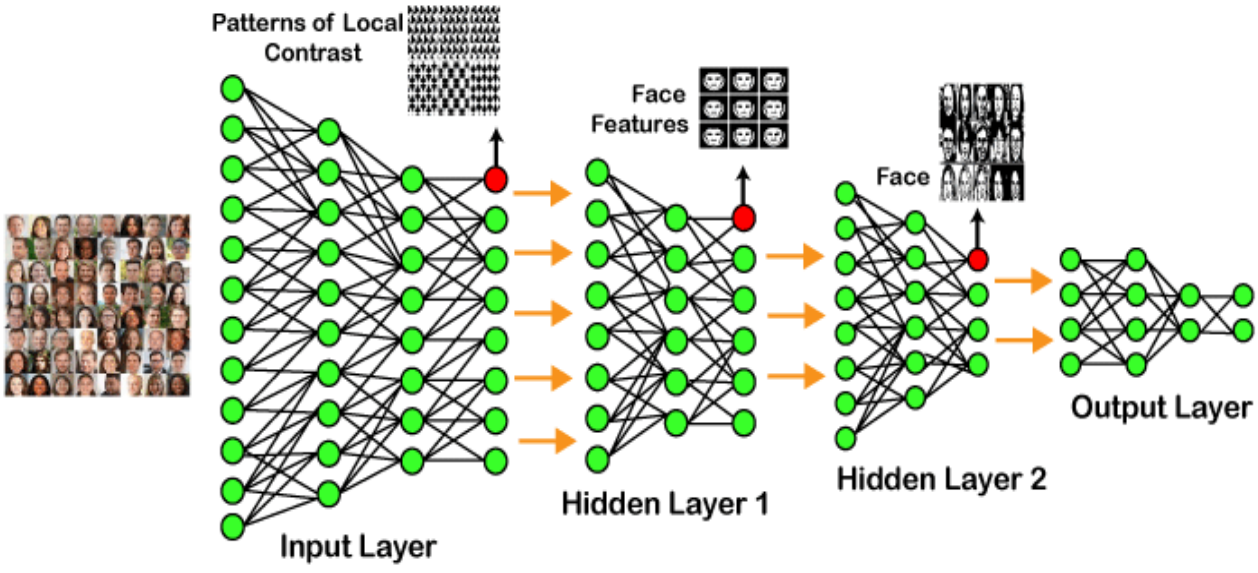
Fig: Boltzmann Machine

Introduction to Deep Learning

Deep learning is based on the branch of machine learning, which is a subset of artificial intelligence. Deep learning is a subset of machine learning in artificial intelligence (AI) that imitate the workings of the human brain in processing data and creating patterns for use in decision making. It involves neural networks with many layers ("deep") that can learn and make intelligent decisions on their own. Basically, it is a machine learning class that makes use of numerous nonlinear processing units so as to perform feature extraction as well as transformation. The output from each preceding layer is taken as input by each one of the successive layers.

[" Deep learning is a collection of statistical techniques of machine learning for learning feature hierarchies that are actually based on artificial neural networks. "]

Example of Deep Learning



In the example given above, we provide the raw data of images to the first layer of the input layer. After then, these input layer will determine the patterns of local contrast that means it will differentiate on the basis of colors, luminosity, etc. Then the 1st hidden layer will determine the face feature, i.e., it will fixate on eyes, nose,

and lips, etc. And then, it will fixate those face features on the correct face template. So, in the 2nd hidden layer, it will actually determine the correct face here as it can be seen in the above image, after which it will be sent to the output layer. Likewise, more hidden layers can be added to solve more complex problems, for example, if you want to find out a particular kind of face having large or light complexions. So, as and when the hidden layers increase, we are able to solve complex problems.

Applications of Neural Network

Facial Recognition: Neural networks, especially Convolutional Neural Networks (CNNs), are used to identify and verify human faces in images and videos.

Stock Market Prediction: They help in predicting stock prices by analyzing historical data and identifying patterns.

Healthcare: Neural networks assist in diagnosing diseases, analyzing medical images, and predicting patient outcomes.

Natural Language Processing (NLP): They are used in applications like language translation, sentiment analysis, and chatbots.

Autonomous Vehicles: Neural networks help in object detection, path planning, and decision-making for self-driving cars.

Gaming: They are used to create intelligent behaviors in non-player characters (NPCs) and to develop game strategies.

Cybersecurity: Neural networks detect anomalies and potential threats by analyzing network traffic and user behavior.